

Mašina za kuglice

Madina je izumila mašinu za pravljenje loptica kako bi zabavila učesnike IOI-a. Unutrašnja struktura mašine je sljedeća:

- Mašina se sastoji od N **čvorova**, indeksiranih od 0 do $N - 1$. Čvor $N - 1$ se naziva **korijen**.
- Za svaki čvor u sa $0 \leq u < N - 1$, **roditelj** čvora u je čvor $P[u]$ sa $P[u] > u$, a čvor u je **dijete** čvora $P[u]$. Korijen nema roditelja. Čvor bez djece naziva se **list**.

Ne znate N ili roditeljski $P[u]$ bilo kojeg čvora u . Umjesto toga, Madina vam kaže M , broj listova, i da su indeksi listova $0, 1, \dots, M - 1$. Vaš zadatak je da odredite unutrašnju strukturu mašine tako što ćete je koristiti.

Svaki čvor mašine može sadržavati najviše jednu **lopticu**, a svaka loptica ima **vrijednost** koja je nenegativan cijeli broj. Kažemo da čvor ima vrijednost x ako sadrži lopticu vrijednosti x . Čvor je **zauzet** ako sadrži kuglu; u suprotnom je **slobodan**. U početku je svaki čvor slobodan.

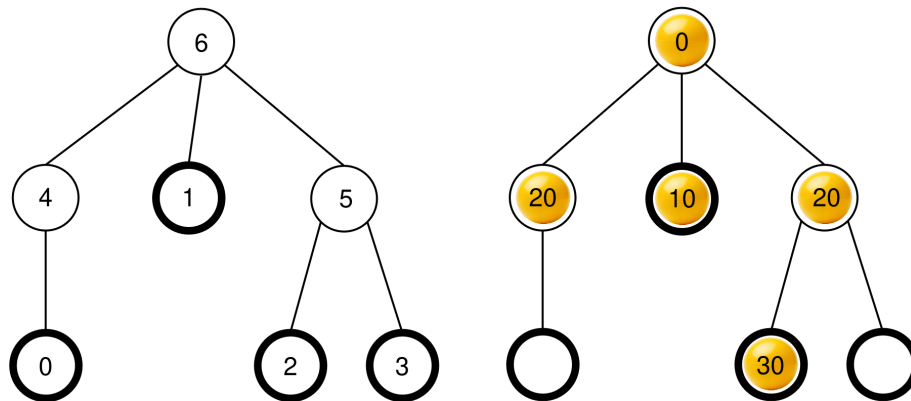
Možete izvršiti dvije vrste operacija:

- **insert/umetni** (U, X): pokušajte da umetnete novu kuglu sa vrijednošću X na list U .
 - Ako je list U zauzet, operacija vraća `false` bez umetanja kuglice.
 - Ako je list U slobodan, kugla se postavlja na čvor U . Zatim se kugla više puta pomiče do roditelja trenutnog čvora sve dok je taj roditelj slobodan. Kretanje se zaustavlja kada kugla dođe do korijena ili čvora čiji je roditelj zauzet. Operacija vraća `true`.
- **collect()**: ako je korijen slobodan (što znači da je mašina prazna), `collect` vraća prazan niz. U suprotnom, rekursivna procedura `traverse` opisana u nastavku sakuplja vrijednosti svih kuglica koje se trenutno nalaze u mašini u niz S . U početku, niz S je prazan i poziva se `traverse(N - 1)`.

```
traverse(u):  
    append the value of the ball at node u to the end of S  
    let c[u] be the list of the occupied children of u  
    sort c[u] in non-decreasing order according to the values  
        of the balls at those nodes (if multiple nodes have the  
        same value, they can appear in any order)  
    for each v in c[u]:  
        traverse(v)
```

Nakon što se ova procedura završi, **sve kuglice se uklanjaju** iz mašine, a `collect` vraća S .

Na primjer, razmotrimo mašinu sa $N = 7$ čvorova i roditeljskim nizom $P = [4, 6, 5, 5, 6, 6]$. Slika lijevo prikazuje praznu mašinu sa indeksima čvorova, dok slika desno prikazuje moguću konfiguraciju kuglica nakon nekog niza operacija `insert` (ovaj niz je dat u odjeljku Primjeri).



Kada se pozove `collect()`, korijen (čvor 6) je zauzet, pa se S inicijalizira kao $S = []$ i poziva se `traverse(6)`.

`traverse(6)` : čvor 6 ima vrijednost 0 ; dodavanjem na S dobija se $S = [0]$. Djeca čvora 6 su 4, 1 i 5, koji su svi zauzeti. Njihove vrijednosti su 20, 10 i 20, respektivno. Sortiranjem u neopadajućem redoslijedu dobija se $c[6] = [1, 5, 4]$ (imajte na umu da bi $c[6] = [1, 4, 5]$ takođe bilo validno, jer čvorovi 4 i 5 imaju istu vrijednost). Procedura zatim poziva `traverse` na svakom čvoru u $c[6]$ tim redoslijedom.

- **`traverse(1)`** : čvor 1 ima vrijednost 10 ; dodavanjem na S dobija se $S = [0, 10]$. Čvor 1 nema zauzetih potomaka (djece), tako da se ovaj poziv završava.
- **`traverse(5)`** : čvor 5 ima vrijednost 20 ; dodavanjem na S dobija se $S = [0, 10, 20]$. Čvor 5 ima jedno zauzeto dijete, čvor 2, tako da je $c[5] = [2]$ i procedura poziva `traverse(2)`.
 - **`traverse(2)`** : čvor 2 ima vrijednost 30 ; dodavanjem na S dobija se $S = [0, 10, 20, 30]$. Čvor 2 nema zauzetih potomaka (djece), tako da se ovaj poziv završava.
 - Izvršenje se vraća na `traverse(5)` koji je obradio svu djecu u $c[5]$, tako da se njegov poziv završava.
- **`traverse(4)`** : čvor 4 ima vrijednost 20 ; dodavanjem na S dobija se $S = [0, 10, 20, 30, 20]$. Čvor 4 nema zauzetih potomaka (djece), tako da se ovaj poziv završava.

Svi pozivi su završeni i nema više čvorova za obradu. Konačno, svaka kuglica se uklanja iz mašine i `collect` vraća niz $S = [0, 10, 20, 30, 20]$.

Vaš zadatak je da odredite broj čvorova N i da prijavite niz roditelja $R = [R[0], R[1], \dots, R[N - 2]]$ koji opisuje strukturu mašine, ali sa potencijalno drugačijim označavanjem čvorova koji nisu listovi

нити korijen. Formalno, prijavljeni niz R se smatra ispravnim ako je moguće dodijeliti različitu oznaku $L[u]$ sa $0 \leq L[u] < N$ svakom čvoru u mašine tako da:

- $L[u] = u$ za sve $0 \leq u < M$ i $u = N - 1$, i
- $L[P[u]] = R[L[u]]$ za sve $0 \leq u < N - 1$.

Nije obavezno da $R[u] > u$. Mašina **mora biti prazna** kada prijavljujete svoje rješenje.

Neka je K broj izvršenih operacija `collect`, a B maksimalna vrijednost na bilo kojoj kuglici u svim pozivima `insert`. Neka je $C = K + B$. Tada C **ne smije biti veći od 1000**. Vaš rezultat u nekim podzadacima zavisi od vrijednosti C .

Detalji implementacije

Trebali biste implementirati sljedeći postupak.

```
std::vector<int> find_structure(int M)
```

- M : broj listova u mašini.
- Procedura se poziva tačno jednom za svaki testni slučaj.
- Procedura mora vratiti niz $R = [R[0], R[1], \dots, R[N - 2]]$ dužine $N - 1$, ispravan niz roditelja.

Da bi komunicirala sa mašinom, vaša procedura može pozvati sljedeće dvije procedure.

Prvi postupak je sljedeći:

```
bool insert(int U, int X)
```

- U : indeks lista u koji treba umetnuti kuglicu. Uslov $0 \leq U < M$ mora biti ispunjen.
- X : cjelobrojna vrijednost loptice. Uslov $0 \leq X \leq 1000$ mora biti ispunjen.
- Ova procedura vraća `true` ako je kuglica uspješno ubačena, ili `false` ako je list U već bio zauzet.
- Ova procedura se može pozvati najviše 500 000 puta.

Drugi postupak je sljedeći:

```
std::vector<int> collect()
```

- Prikuplja vrijednosti na svim ubačenim kuglicama, prazni mašinu i vraća prikupljeni niz S .

Ako u bilo kojem trenutku tokom izvršavanja vašeg programa vrijednost C premaši 1000, vaše rješenje će dobiti presudu `Output isn't correct: Too many resources used`.

Struktura mašine je fiksirana prije nego što se pozove `find_structure`. Grader je **deterministički** u smislu da ako ga pokrenete dva puta i oba pokretanja izvrše isti niz operacija, pozivi funkcije `collect` će vratiti iste nizove.

Ograničenja

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podzadaci

Podzadatak	Rezultat	Dodatna ograničenja
1	5	Korijen ima tačno M djece.
2	10	$M \leq 3$
3	$\$25N \leq 200, M \leq 45\$$	
4	60	Nema dodatnih ograničenja.

U podzadacima 3 i 4, vaš rezultat zavisi od vrijednosti C na sljedeći način.

Podzadatak 3 (25 bodova)

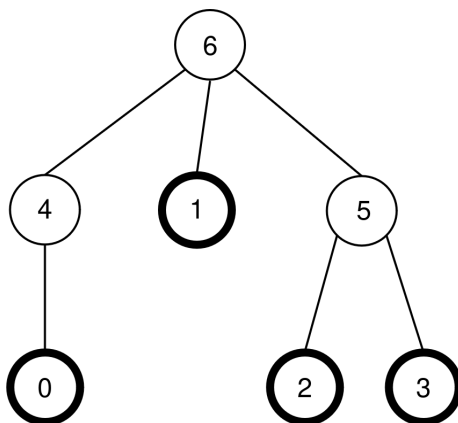
Limiti	Rezultat
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podzadatak 4 (60 bodova)

Limiti	Rezultat
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Primjer

Razmotrimo istu mašinu kao u opisu zadatka, dakle $N = 7$, $M = 4$ i $P = [4, 6, 5, 5, 6, 6]$. Mašina je ponovo prikazana na sljedećoj slici:



Ocjenjivač donosi sljedeću odluku:

```
find_structure(4)
```

Procedura može izvršiti sljedeći niz poziva:

1. **insert(0, 0)** : list 0 je slobodan, pa se kuglica s vrijednošću 0 postavlja na list 0 . Čvor $P[0] = 4$ je slobodan, pa se kuglica pomiče na čvor 4 . Čvor $P[4] = 6$ je slobodan, pa se kuglica pomiče na čvor 6 . Pošto je čvor 6 korijen, kretanje se zaustavlja. Poziv vraća `true` .
2. **insert(3, 20)** : list 3 je slobodan, pa se kuglica vrijednosti 20 postavlja na list 3 . Čvor $P[3] = 5$ je slobodan, pa se kuglica pomiče na čvor 5 . Pošto je $P[5] = 6$ zauzet, kretanje se zaustavlja. Poziv vraća `true` .
3. **insert(1, 10)** : list 1 je slobodan, pa se kuglica vrijednosti 10 postavlja na list 1 . Pošto je $P[1] = 6$ zauzeto, kuglica ostaje na čvoru 1 . Poziv vraća `true` .
4. **insert(2, 30)** : list 2 je slobodan, pa se kuglica vrijednosti 30 postavlja na list 2 . Pošto je $P[2] = 5$ zauzeto, kuglica ostaje na čvoru 2 . Poziv vraća `true` .
5. **insert(1, 25)** : list 1 je zauzet, tako da loptica nije postavljena na list 1 . Poziv vraća `false` .
6. **insert(0, 20)** : list 0 je slobodan, pa se kuglica vrijednosti 20 postavlja na list 0 . Čvor $P[0] = 4$ je slobodan, pa se kuglica pomiče na čvor 4 . Pošto je $P[4] = 6$ zauzet, kretanje se zaustavlja. Poziv vraća `true` .

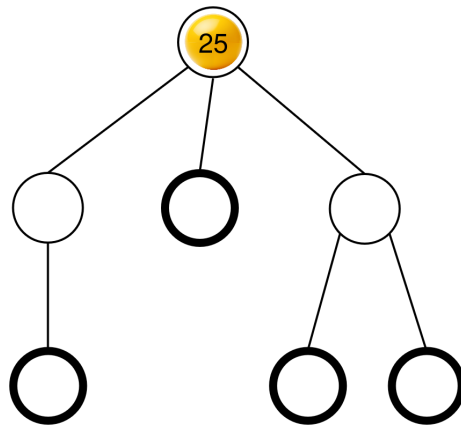
Rezultirajuća konfiguracija kuglica je već bila prikazana u opisu zadatka.

Zatim, procedura može pozvati:

```
collect()
```

Izvršavanje ovog poziva je već objašnjeno u opisu zadatka, tako da ovaj poziv vraća $S = [0, 10, 20, 30, 20]$. Nakon toga, mašina je ponovo prazna.

Zatim, procedura može pozvati `insert(2, 25)`. List 2 je slobodan, pa se kuglica vrijednosti 25 postavlja na list 0. Ova kuglica se zatim pomiče do čvora 5, a zatim do čvora 6, gdje se zaustavlja. Poziv vraća `true`. Rezultirajuća konfiguracija kuglica prikazana je na sljedećoj slici.



Konačno, procedura može pozvati `collect()` koja vraća $S = [25]$ i prazni mašinu.

Postoje dva ispravna roditeljska niza koja `find_structure` može vratiti: $R = P = [4, 6, 5, 5, 6, 6]$ (što odgovara oznaci $L = [0, 1, 2, 3, 4, 5, 6]$) i $R = [5, 6, 4, 4, 6, 6]$ (što odgovara oznaci $L = [0, 1, 2, 3, 5, 4, 6]$). U ovom primjeru, $K = 2$ i $B = 30$, tako da je $C = 32$.

Primjer ocjenjivača

Format unosa:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Izlazni format:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Ovdje je Q dužina niza R kojeg vraća funkcija `find_structure`.